TIA — Technical Framework Document

Enterprise Change Impact Intelligence Platform

Version	1.3
Status	Proposed technical baseline
Document set	PRD.md v3.1 · ARCHITECTURE.md v1.3 · TECHNICAL-FRAMEWORK.md v1.3 · UI-PRD.md (referenced; not part of this set)
Primary domain	Banking and financial-services engineering
Architecture style	Modular, evidence-driven, API-first platform
Last updated	2026-10-28

> **Precedence.** This document owns the canonical data model (§5), status enums (§5.3, §7), inference rules (§5.4), identity-resolution rules (§5.5), and engineering standards. Scope, priorities, and phases are owned by PRD.md; component boundaries and data flow by ARCHITECTURE.md.

---

1. Purpose

The Technical Framework Document defines the engineering framework for implementing TIA. It establishes the technology baseline, system boundaries, internal modules, graph model, processing contracts, APIs, evidence standards, security controls, deployment approach, and implementation sequence.

TIA must answer:

> Given a proposed code, API, event, or data-schema change, which downstream components may be affected, what evidence supports each finding, what verification is required, and what remains unknown?

The system must provide explainable, reproducible technical impact analysis rather than simply displaying dependencies or generating an AI-written summary.

---

2. Architectural principles

1. **Evidence before conclusions:** Every material impact finding must link to supporting evidence.
2. **Deterministic analysis:** Dependency traversal, revision comparison, severity, and policy evaluation must be reproducible for the same inputs, configuration, and evidence snapshot.
3. **Explicit uncertainty:** Missing, stale, inferred, and incomplete evidence must remain distinguishable.
4. **Version-aware relationships:** Dependencies must be associated with relevant source revisions, contract versions, and observation windows.

5. **Incremental processing:** A new change should update affected evidence instead of requiring a full rebuild unnecessarily.
6. **Separation of concerns:** Source extraction, identity resolution, graph persistence, impact analysis, and policy evaluation are separate responsibilities.
7. **Ingest, then analyse:** External systems are read by connectors into TIA's stores. Analysis reads only from those stores, pinned to an evidence snapshot. Analysis never calls external systems.
8. **No false completeness:** A successful scan of one repository does not establish complete knowledge of the enterprise.
9. **Enterprise security from day one:** Access to source code, evidence, analysis reports, and administrative operations must be authorised and audited from Release 1.

---

3. Reference technology stack

The following is the proposed baseline, not a claim that these technologies have already been implemented or benchmarked.

Layer	Proposed technology	Responsibility
User interface	React, TypeScript	Impact reports, evidence exploration, service views (Release 2)
API layer	Python, FastAPI, Pydantic	API contracts, validation, authorisation, integration
Source analysis — Java	Tree-sitter Java grammar + Spring adapters (`@RestController`/`@RequestMapping`, `RestTemplate`, `WebClient`, OpenFeign, JPA entities, Flyway/Liquibase migrations)	Symbols, calls, endpoints, outbound call sites, data access
Source analysis — Python	Python `ast` / Tree-sitter + FastAPI / Flask / Django REST adapters (`requests`, `httpx`, SQLAlchemy, Alembic)	Same as above
Contract analysis	OpenAPI parsers and schema-diff utilities	API and schema change detection
Metadata database	PostgreSQL	Repositories, jobs, revisions, policies, analysis snapshots
Dependency graph	RDF graph store or property graph (benchmark first)	Typed dependencies and evidence-backed traversal
Evidence storage	S3-compatible object storage or controlled filesystem	Source references, manifests, reports, scan artifacts
Background execution	PostgreSQL-backed job queue initially	Asynchronous scans and analysis jobs
Source integration	GitHub App and webhooks	PR events, commits, repository access, PR comments/checks
Testing	pytest and API/integration tests	Extraction, graph, impact, and policy verification
Delivery	Containerised services and CI/CD	Repeatable build, deployment, and release

Java and Python adapters are both delivered in Release 1; further languages ship through the adapter SDK in Release 4 (PRD FR-040).

Technology decision rule

Do not introduce Kafka, Kubernetes, a vector database, multiple graph databases, or an LLM orchestration framework by default. Add infrastructure only when measured scale, availability, or operational requirements justify it.

The graph-store choice must be validated against representative queries, update frequency, traversal depth, evidence requirements, and deployment constraints before committing to a production implementation. For Release 1, PostgreSQL with versioned adjacency tables and recursive queries is the graph store; the Release 2 benchmark decides whether a dedicated graph store is needed at enterprise scale (PRD §12).

---

4. Logical system architecture

The platform is organised into six logical modules. Their runtime boundaries and data flow are defined in ARCHITECTURE.md §3–§5.

4.1 Connector module

Responsibilities:

- Register repositories and source-control integrations.
- Retrieve PR metadata, commit SHAs, diffs, and permitted source content.
- Retrieve contract specifications, CI results, test reports, deployment metadata, runtime observations, and ITSM change-record references when connected.
- Write retrieved artifacts to the evidence store with collection timestamps.
- Record connector permissions, failures, and source coverage.
- Publish outputs (PR comments/checks, ITSM attachments) on behalf of the reporting module.

Each connector must expose consistent status and error information.

4.2 Extraction module

Responsibilities:

- Parse source files and identify symbols, functions, classes, endpoints, imports, and supported call relationships.
- Identify **outbound call sites** (HTTP client calls, generated-client invocations, message producers) with their path templates and the configuration keys that supply base URLs.
- Identify API and event contracts.
- Extract test definitions and test-to-code evidence where available.
- Extract data-access relationships from ORM mappings and migration files (raw SQL and stored procedures are covered by PRD FR-041 in Release 4).
- Preserve source locations and revision identifiers.

Extraction emits **normalised entities and evidence records** to the identity-resolution module. It never writes graph edges directly.

Extraction must be language-adapter based; see §3 for the adapters.

4.3 Identity resolution and graph writer

Responsibilities:

- Resolve entities across repositories and revisions.
- Resolve outbound call sites to provider endpoints using only the strategies in §5.5.
- Associate endpoints with service identities and contract definitions.
- Normalise identifiers and relationship types.
- Store provenance, evidence status, and temporal validity.
- Invalidate or supersede stale relationships when source evidence changes.
- Prevent unverified candidate relationships from silently becoming authoritative.

The identity-resolution module is the **only writer** to the graph store. It must distinguish an entity's identity from the evidence supporting a relationship to another entity.

4.4 Impact-analysis engine

The engine has **read-only** access to the graph store, evidence store, and metadata database. It never writes to the graph and never calls connectors.

Responsibilities:

1. Resolve the base and proposed revisions and pin the evidence snapshot.
2. Identify changed entities.
3. Find directly related components.
4. Traverse relevant dependency relationships within configured bounds.
5. Correlate affected components with tests, contracts, deployments, and runtime evidence already in the stores.
6. Classify findings by impact type, severity (§7.2), evidence status, and limitations.
7. Record unresolved dependencies, unresolved outbound call sites, and analysis coverage.
8. Produce an immutable analysis snapshot.

The engine must not interpret a missing graph edge as proof that a dependency does not exist.

4.5 Policy engine

Responsibilities:

- Evaluate configured release and verification policies against an analysis snapshot.
- Determine whether mandatory evidence or tests are missing.
- Apply policy scope, conditions, severity, and outcome.
- Return explicit results: `pass`, `fail`, `not_evaluated`, or `requires_review`.
- Record the policy version and evidence used for each decision.

A policy evaluation must not convert an incomplete analysis into a pass.

4.6 Reporting and API module

Responsibilities:

- Orchestrate analysis jobs (ingest → resolve → analyse → evaluate → publish).
- Provide analysis summaries and detailed findings.

- Serve impacted-component lists and evidence paths.
- Expose test, contract, runtime, and deployment evidence.
- Return unknowns, limitations, and coverage information.
- Render the PR comment and, from Release 2, the PR check and ITSM attachment.
- Filter findings by the requesting user's repository permissions.
- Support UI rendering and export.

These modules may initially run in one deployable backend with clear internal interfaces. Separate services should be introduced only when independent scaling or isolation is required.

---

5. Canonical technical data model

5.1 Core entities

Entity	Purpose	Release
Repository	Source-control repository	1
Commit	Immutable source revision	1
File	Source file at a revision	1
FunctionVersion	Function or method at a particular revision	1
Service	Deployable or logical application service	1
APIEndpoint	Exposed API operation (method + path template)	1
OutboundCallSite	A location in source that issues a call to another service	1
Schema	Versioned API, event, or data schema	1
DataEntity	Logical business or technical data entity	1
Table	Physical database table	1
Migration	Versioned database migration	1
TestCase	Defined verification test	1
Owner	Responsible team or organisational identity	1
EvidenceRecord	Source supporting an entity or relationship	1
EvidenceSnapshot	Immutable set of evidence-record versions used by an analysis	1
AnalysisRun	Immutable impact-analysis execution	1
Event	Published or consumed event / topic	2
TestRun	Execution of a test in a specific build or run	2
BuildArtifact	Built, versioned artifact	2
Deployment	Release of an artifact into an environment	2
Environment	Development, test, staging, production, etc.	2
Policy	Versioned verification or release rule	2
ChangeRecord	Reference to an external ITSM change record	2

Entity	Purpose	Release
Incident	Operational incident record	3

5.2 Core relationship types (canonical)

This is the single relationship vocabulary for TIA. PRD.md and ARCHITECTURE.md reference it; the implementation must use these names.

Relationship	Direction	Meaning	Release
`CONTAINS`	Repository/File → File/FunctionVersion	Structural containment	1
`CHANGED_IN`	FunctionVersion/File/Schema → Commit	Entity was modified in the commit	1
`CALLS`	FunctionVersion → FunctionVersion	In-process call extracted from source	1
`HANDLES`	FunctionVersion → APIEndpoint	Function is the handler for the endpoint	1
`EXPOSES`	Service → APIEndpoint	Service exposes the endpoint	1
`ISSUES`	FunctionVersion → OutboundCallSite	Function contains the outbound call site	1
`CONSUMES_API`	OutboundCallSite → APIEndpoint	**Synchronous** API consumption, produced only by §5.5 strategies	1
`USES_SCHEMA`	APIEndpoint/Event/FunctionVersion → Schema	Uses the schema for request, response, or payload	1
`READS`	FunctionVersion → DataEntity/Table	Reads data (ORM/migration-declared; SQL in Release 4)	1
`WRITES`	FunctionVersion → DataEntity/Table	Writes data (ORM/migration-declared; SQL in Release 4)	1
`MIGRATES`	Migration → Table	Migration changes table structure	1
`TESTS`	TestCase → FunctionVersion/APIEndpoint/Service/Schema	Test is statically associated with the target	1
`OWNED_BY`	Service/Repository → Owner	Assigned owner	1
`PRODUCES_EVENT`	FunctionVersion/Service → Event	**Asynchronous** publication	2
`CONSUMES_EVENT`	FunctionVersion/Service → Event	**Asynchronous** consumption	2
`EXECUTED_IN`	TestCase → TestRun	Test executed in the run	2
`COVERS`	TestRun → FunctionVersion	Coverage evidence associates execution with code	2
`BUILT_FROM`	BuildArtifact → Commit	Artifact was built from the revision	2
`DEPLOYED_AS`	Service → Deployment	Service is deployed as the deployment	2
`DEPLOYED_TO`	Deployment → Environment	Deployment runs in the environment	2
`OBSERVED_CALL_TO`	Service → Service/APIEndpoint	Runtime telemetry recorded a call	2

Relationship	Direction	Meaning	Release
`GOVERNED_BY`	Service → Policy	Policy applies to the service	2
`ASSOCIATED_WITH_INCIDENT`	Service/Deployment/Commit → Incident	Incident association, with association type (PRD FR-031)	3

These relationships have explicit, non-overlapping semantics. In particular:

- `CALLS` (source, in-process) ≠ `CONSUMES_API` (source, cross-service) ≠ `OBSERVED_CALL_TO` (runtime).
- `CONSUMES_API` (synchronous) ≠ `CONSUMES_EVENT` (asynchronous).
- `TESTS` (static association) ≠ `EXECUTED_IN` (test ran) ≠ `COVERS` (test executed code).

5.3 Evidence and provenance model

Every important relationship carries, directly or through a linked evidence record:

- Evidence identifier and evidence type.
- Source repository, file, line range, or external record.
- Observed revision or artifact version.
- Collection and analysis timestamps.
- Validity interval where applicable.
- Extraction method and tool version.
- Resolution strategy (§5.5) or inference rule (§5.4), where applicable.
- Evidence status.
- Freshness and known limitations.

Evidence status values (lower snake case everywhere — graph, API, report, UI):

Value	Meaning
`source_confirmed`	Supported by an explicit source-code or configuration relationship
`contract_confirmed`	Supported by an API, event, or schema definition
`runtime_observed`	Observed in available runtime telemetry
`corroborated`	Supported by two or more independent evidence sources of different kinds
`inferred`	Produced by a named rule in §5.4; requires validation
`unknown`	Insufficient evidence to determine the relationship
`stale`	Evidence may no longer describe the current revision or deployment

These statuses describe the nature of the evidence, not an absolute guarantee of correctness.

5.4 Inference rules

`inferred` relationships may only be produced by the versioned rules below. Each inferred relationship records `inference_rule` and rule version. New rules require a framework revision and golden-corpus tests (§13).

Rule	Produces	Condition	Never applies when
`INF-PATH-UNIQUE`	`CONSUMES_API`	Outbound call site has a literal/constant path template that matches **exactly one** in-scope provider endpoint (method + normalised template), but the base URL cannot be resolved	Template matches 0 or ≥ 2 endpoints; path built entirely at runtime
`INF-TEST-NAMING`	`TESTS`	Test name/location follows a configured naming convention pointing at the target (e.g. `PaymentServiceTest` → `PaymentService`) and no import/structural association exists	The convention is not configured for the repository
`INF-SCHEMA-SHARED`	`USES_SCHEMA`	Consumer DTO is structurally identical (field names and types) to a provider schema **and** the consumer has a `CONSUMES_API` relationship to an endpoint using that schema	No `CONSUMES_API` exists between the two

Not inference (forbidden as relationship sources): name or string similarity alone, LLM output, co-occurrence in documentation, and temporal proximity alone.

5.5 Cross-repository identity resolution

Resolution of an `OutboundCallSite` to an `APIEndpoint` (`CONSUMES_API`) uses only the strategies below, applied in order. The first strategy that succeeds determines the status; later successful strategies add corroborating evidence (→ `corroborated` when the kinds differ).

ID	Strategy	Inputs	Status
R1	Generated or shared client	Consumer imports a client package generated from, or published by, the provider repository (package coordinates mapped to provider)	`source_confirmed`
R2	Fully resolved HTTP call	Call site path template + base-URL configuration key; key value resolves to a service via R4, a gateway route table, or a service registry export in source control	`source_confirmed`
R3	Declared contract	Consumer-side OpenAPI reference or consumer-driven contract file (e.g. Pact) naming provider and interaction	`contract_confirmed`
R4	Service map	`tia-services.yaml` (owner-maintained, in source control) mapping hostnames / config keys / package coordinates to services	Input to R1/R2 only
R5	Unique path match	See `INF-PATH-UNIQUE`	`inferred`
R6	Runtime observation (Release 2)	Trace span from consumer service to provider endpoint	`runtime_observed`

Outputs per analysis:

- Every unresolved call site, with file and line, listed as an unknown.
- Resolution rate overall and per strategy (PRD FR-014). Formula: `resolved_call_sites / detected_call_sites` over the analysed repositories.

Path-template normalisation: lower-case static segments; replace path variables of any syntax (`{id}`, `:id`, `<id>`, `${id}`) with `{}`; strip trailing slashes and query strings; apply configured gateway prefix rewrites.

---

6. Change-analysis processing contract

The primary workflow is a proposed code change submitted through a pull request. Steps 1–5 are **ingestion** (may call external systems); steps 6–12 are **analysis** (read-only against TIA stores).

Ingestion

1. **Accept the request:** Record the repository, PR, base SHA, proposed SHA, optional change-record id, and analysis configuration.
2. **Validate inputs:** Confirm revision availability, repository permissions, and required integration status.
3. **Extract the diff:** Identify changed files, symbols, endpoints, outbound call sites, schemas, and other supported entities.
4. **Update evidence:** Reuse valid unchanged evidence and refresh affected evidence (including any newly available CI/runtime/deployment evidence already collected by connectors).
5. **Resolve identities:** Connect changed entities to known components and dependency relationships (§5.5); write the graph.

Analysis

6. **Pin the snapshot:** Create an `EvidenceSnapshot` referencing the exact evidence-record versions to be used.
7. **Traverse dependencies:** Follow supported relationships using configured depth and scope limits.
8. **Correlate verification:** Identify associated tests, coverage, contract checks, runtime observations, and deployments present in the snapshot.
9. **Classify findings:** Determine the affected component, impact category, evidence status, severity (§7.2), materiality, and limitations.
10. **Evaluate policies:** Apply the configured rules to the snapshot and analysis coverage (Release 2).
11. **Persist the result:** Save the analysis inputs, snapshot id, outputs, evidence references, policy version, and limitations.
12. **Publish the report:** Return results to the API and UI, and publish the PR comment (Release 1) / check and ITSM attachment (Release 2) via the connector module.

An analysis must identify the exact revisions evaluated. If the proposed revision changes while analysis is running, the result must remain associated with the original revision pair and must not be presented as an assessment of the newer change; the PR comment is updated only by the analysis of the newest revision.

---

7. Impact-analysis result contract

7.1 Fields

Field	Description
`analysis_id`	Unique analysis identifier
`base_revision`	Exact starting revision
`proposed_revision`	Exact proposed revision
`evidence_snapshot_id`	Snapshot used for the analysis
`status`	See §7.3
`coverage`	Sources and evidence successfully evaluated; resolution rate by strategy
`findings`	Impacted components and supporting evidence

Field	Description
`unknowns`	Unresolved dependencies, unresolved call sites, and missing evidence
`policy_results`	Individual policy outcomes
`generated_at`	Analysis creation timestamp
`configuration_version`	Rules, limits, and configuration used (including severity-rule and inference-rule versions)
`extractor_versions`	Extractor and adapter versions used

Each finding includes: component identifier, impact type, severity, `severity_rule`, evidence status, `resolution_strategy` / `inference_rule` where applicable, relationship path, source references, associated test evidence, limitations, recommended verification checks, and `material` (boolean).

Recommendations must be traceable to evidence and rules. They must not be presented as proof that a test is mandatory unless the applicable policy establishes that requirement.

7.2 Severity rules

Severity is computed deterministically by the versioned default rules in PRD §7.3 (inputs: change nature on the path × affected-component criticality; inferred-only paths capped at Medium). Each finding records the rule id that set its severity (e.g. `SEV-BREAKING-CRITICAL`, `SEV-BREAKING-CONFIRMED`, `SEV-NONBREAKING-CRITICAL`, `SEV-NONBREAKING-CONFIRMED`, `SEV-INFERRED-CAP`, `SEV-LOCAL`, `SEV-UNCLASSIFIED`, `SEV-POLICY-FAIL`).

7.3 Analysis status (canonical)

Value	Meaning
`running`	In progress; no conclusions may be shown
`complete`	All in-scope sources analysed; no limits reached; no unknowns affecting material findings
`complete_with_gaps`	Ran to completion; unknowns, unresolved call sites, or missing optional sources exist
`partial`	A limit was reached, evidence was stale, or an in-scope repository/source failed
`failed`	No findings could be produced (e.g. base/head unavailable, changed component unresolvable)

8. API framework

Proposed REST endpoints:

Endpoint	Purpose
`POST /api/v1/repositories`	Register a repository
`POST /api/v1/scans`	Submit a source or evidence scan
`GET /api/v1/scans/{scan_id}`	Retrieve scan status and coverage
`POST /api/v1/impact-analyses`	Create an impact analysis
`GET /api/v1/impact-analyses/{analysis_id}`	Retrieve the report
`GET /api/v1/impact-analyses/{analysis_id}/paths`	Retrieve evidence-backed impact paths

Endpoint	Purpose
`GET /api/v1/impact-analyses/{analysis_id}/unknowns`	Retrieve unknowns and unresolved call sites
`GET /api/v1/components/{component_id}/dependencies`	Retrieve component dependencies
`GET /api/v1/policies`	List available policies (Release 2)
`POST /api/v1/policy-evaluations`	Evaluate policies against an analysis (Release 2)

API conventions:

- Validate request and response models.
- Use stable error codes and structured error responses.
- Make job submission idempotent where appropriate (keyed by repository + base SHA + proposed SHA + configuration version).
- Return job identifiers for asynchronous processing.
- Apply pagination and traversal limits.
- Enforce authorisation at the API boundary and on underlying resources; findings in repositories the caller cannot read are redacted to a count.
- Version externally consumed contracts.

---

9. Persistence and consistency

9.1 Multi-tenancy (PRD 11.2, PL-003)

- Every tenant-owned table carries `tenant_id` (UUID, `NOT NULL`) as the first column of its primary key and of every index used for lookups.
- Row-level security is **enabled and forced** on every tenant-owned table. The single policy is `tenant_id = current_setting('tia.tenant_id')::uuid` for `USING` and `WITH CHECK`.
- The application connects as a role **without** `BYPASSRLS` and without table ownership. Each transaction runs `SET LOCAL tia.tenant_id = '<uuid>'` before any statement; a connection with no tenant set sees no rows.
- Only the tenant-administration service (provisioning, offboarding, metering) uses a separate role that can see the `tenants` registry; it never reads tenant evidence.
- Background jobs carry the tenant id and set it the same way; the queue itself is tenant-scoped, with per-tenant concurrency limits (PL-017).
- Evidence objects (artifact store) are stored under a per-tenant prefix and encrypted with a per-tenant data key (PL-016 adds customer-managed keys).
- Dedicated and customer-hosted deployments run the same schema with a single tenant; isolation code is never disabled.

9.2 Stores and consistency

Use PostgreSQL for transactional metadata, job state, policy definitions, and analysis records. Use a graph store for typed relationships and bounded dependency traversal. Store large evidence artifacts outside the graph and reference them through stable identifiers.

The implementation must define:

- A canonical identifier strategy.

- Transaction boundaries for graph and metadata updates.
- Retry and duplicate-processing behaviour.
- Idempotent ingestion operations.
- Rules for superseding or invalidating stale evidence.
- How evidence snapshots are materialised (versioned evidence records referenced by id, not copied).
- Recovery behaviour when one persistence layer succeeds and another fails.
- Retention and deletion rules for source-derived evidence.

Graph updates and relational updates must not be assumed to be atomic across separate systems. Use explicit processing states, retryable operations, and reconciliation checks.

---

10. UI integration framework

The UI consumes the API rather than independently reconstructing impact logic. Detailed screen requirements are in UI-PRD.md (not part of this document set).

Release 1 surface: the PR comment and CLI output.

Release 2+ principal screens:

1. Overview.
2. Change Impact Report.
3. Impact Path Explorer.
4. Services.
5. Contracts.
6. Tests and Evidence.
7. Deployments and Incidents.
8. Policies.
9. Administration.

The report must expose the finding, supporting evidence, dependency path, verification status, and unresolved questions together.

Every surface (PR comment, CLI, UI) must differentiate:

- No findings from failed analysis.
- Missing evidence from a failed test.
- A test's existence from a test's execution and pass status.
- Source-confirmed from inferred from runtime-observed dependencies.
- A policy pass from a policy that was not evaluated.

Graph visualisation is a supporting interaction, not the sole representation of analysis results. Users must also be able to inspect findings in accessible tables and evidence panels.

---

11. Security and governance

The following are required in Release 1 (PRD §11 P0, PL-001, PL-002, PL-011), not deferred to hardening:

- Least-privilege repository and integration permissions (GitHub App: contents/metadata read; pull-request write only for comments).
- Role-based access to analysis, administration, and policy management.
- Per-user repository-permission filtering of findings.
- Secret storage through an approved secrets manager.
- Encryption in transit and at rest.
- Audit records for ingestion, analysis, exports, external writes, and administrative changes.
- Controlled outbound network access.
- Protection against malicious repository content and unsafe parser execution (parsers never execute repository code or build scripts).
- Explicit handling of credentials, secrets, and sensitive data discovered in source files (never copied into graph or reports).

Release 2–3:

- Configurable retention and deletion of source-derived evidence.
- Tenant or organisation isolation where required.

The deployment model must support a private environment for banks that cannot permit source code or engineering metadata to leave their controlled boundary.

---

12. Reliability and performance

The system must be designed and tested for:

- Large repositories and multi-repository dependency traversal.
- Partial connector failures.
- Parser failures and unsupported language constructs.
- Duplicate or delayed webhook events.
- Stale evidence and changing source revisions.
- Interrupted background jobs.
- Graph traversal limits and incomplete result sets.
- Concurrent analysis requests.
- Retry and recovery after persistence failures.

Performance targets are in PRD §12 and should be validated on a representative bank application corpus. Measure scan duration, impact-report latency, memory use, graph-query cost, and concurrent analysis capacity.

Do not advertise enterprise-scale performance until those measurements have been collected.

---

13. Testing framework

Unit tests. Validate parsers, identifier and path-template normalisation, relationship extraction, resolution strategies, inference rules, severity rules, evidence status classification, and policy rules.

Integration tests. Validate connectors, database persistence, graph writes, graph traversal, and API responses.

Golden-corpus tests. Maintain representative multi-repository source corpora (per supported language) with known dependency relationships, including deliberately unresolvable call sites and ambiguous path matches. Compare expected and actual findings, statuses, and resolution rates across releases.

Regression tests. Verify that parser or rule changes do not silently remove previously established dependencies or change policy outcomes.

Determinism tests. Re-run analyses on a fixed evidence snapshot and assert identical structured output.

Security tests. Test authorisation boundaries (including cross-repository redaction), secret handling, malicious input, dependency vulnerabilities, and audit behaviour.

End-to-end tests. Submit a representative PR, run the analysis, inspect the evidence paths, and verify the resulting PR comment, report, and (Release 2) policy outcomes.

Evaluate recall, precision, evidence completeness, false-positive rate, noise (material findings per PR), resolution rate, and analysis coverage separately. A single accuracy score is insufficient.

---

14. Observability and operations

Record structured logs, metrics, traces, and job events for:

- Connector success and failure.
- Extraction duration and parser errors.
- Entities and relationships created or superseded.
- Identity-resolution failures and resolution rate by strategy.
- Graph traversal size and duration.
- Analysis completeness and unknown counts.
- Policy outcomes.
- Queue depth, retries, and processing failures.
- Evidence freshness and source coverage.

Operational dashboards must distinguish platform health from analysis completeness. A healthy service can still produce an incomplete report because an external source is unavailable.

---

15. Implementation releases

Releases, scope, and quality gates are defined **only** in PRD §13 and §19. Every release is production quality. This section maps them to engineering work.

PRD release	Engineering work
Release 1	Multi-tenant PostgreSQL schema with forced row-level security and migrations; tenant lifecycle and per-tenant fair scheduling; persisted versioned graph with incremental invalidation (stale handling); job queue; evidence snapshots; GitHub App connector (webhooks, fetch, PR comment); OIDC/SAML SSO, RBAC, SCM permission mirroring; secrets and encryption; Java and Python adapters incl. ORM/migrations; resolution R1–R5; traversal; basic contract diff; severity; report; platform observability; container images, Helm chart, Compose; reference landscape and accuracy harness

PRD release	Engineering work
Release 2	Metering and plan limits; data residency; per-tenant keys / BYOK; dedicated SaaS; CI and coverage ingestion; full contract diff; event adapters; telemetry ingestion (R6); deployments and environments; ownership/criticality registry; policy engine with advisory/enforcement; PR check; ITSM connector; MCP server; web UI and admin console
Release 3	Temporal queries and release comparisons; incident association; retention enforcement and audit reporting; customer-hosted / air-gapped packaging; SOC 2 / ISO 27001 controls; HA and horizontal scale; backup/DR; air-gapped install; public API + SDK
Release 4	GitLab, Bitbucket, Azure DevOps connectors; adapter SDK + C#, TypeScript, Go, Kotlin adapters; SQL/stored-procedure lineage; event-stream lineage; risk trends; calibration; multi-team coordination

Load/resilience testing, security review, and documentation are part of every release's definition of done (PRD §13).

---

16. Acceptance criteria

The technical framework is considered implemented for a phase when the PRD acceptance criteria for that phase (PRD §14) pass, and:

1. An analysis is tied to exact base and proposed revisions and an evidence snapshot.
2. Changed source entities can be resolved to supported graph entities.
3. Impact traversal returns typed, inspectable relationship paths using only §5.2 relationship types.
4. Material findings expose their supporting evidence, resolution strategy, and severity rule.
5. Evidence provenance and validity are preserved.
6. Missing, stale, and inferred relationships are explicitly represented; every inferred relationship names a §5.4 rule.
7. Partial and failed analyses cannot masquerade as complete zero-impact reports.
8. Test existence, execution, result, and coverage remain distinct.
9. Policy evaluations record the policy version and outcome.
10. Results are available through documented APIs, the CLI, and the PR comment (UI from Release 2).
11. Ingestion and analysis operations support retries and duplicate handling.
12. Re-running an analysis on the same snapshot yields identical structured output.
13. Security, regression, and performance tests cover the release scope.

---

17. Technical decisions still requiring validation

Decided in v1.1–v1.2 (subject to reference-landscape evidence):

- Canonical relationship vocabulary (§5.2), evidence statuses (§5.3), analysis statuses (§7.3).
- Inference rules (§5.4) and resolution strategies (§5.5).
- Ingest-then-analyse with evidence snapshots (§2, §6).
- Default severity rules (PRD §7.3).

Still to decide before the production baseline is frozen:

- RDF versus property graph versus PostgreSQL-only, based on representative workload tests (Release 2).
- Canonical identifier format for functions, endpoints, schemas, and services across revisions.
- Evidence freshness thresholds (when evidence becomes `stale`).

- Required depth and scale limits for impact traversal.
- The initial release-policy language and execution semantics.
- Runtime and deployment integration scope.
- Data retention, tenant isolation, and private deployment requirements.
- Reference-landscape corpus composition and baseline measurements.

---

18. Definition of success

TIA succeeds when engineering teams can answer, with inspectable evidence:

- What changed?
- Which components may be affected?
- Why does TIA believe they are affected?
- Which tests and contract checks provide verification?
- Which production observations are relevant?
- What evidence is missing or stale?
- Which release policies passed, failed, or require review?

Core engineering principle: Every important impact finding must explain what is affected, why the relationship is credible, where the evidence comes from, and what remains unknown.

---

19. Document history

Version	Date	Notes
1.0	2026-10-28	Initial technical baseline
1.1	2026-10-28	Aligned with PRD v2.1: precedence statement; ingest-then-analyse principle and evidence snapshots; engine read-only, graph writer sole writer; Java adapter added; OutboundCallSite entity; canonical relationship vocabulary incl. `CONSUMES_API` (was missing) and split sync/async consumption; snake_case evidence statuses; inference rules; cross-repo resolution strategies and path normalisation; canonical analysis status incl. `complete_with_gaps`; severity rule ids; security baseline moved to Phase 1; phases mapped to PRD (pilot evaluated per gate, not at the end); determinism tests; UI-PRD.md marked as outside this set
1.2	2026-10-29	Aligned with PRD v3.0 enterprise scope: phases replaced by Releases 1–4; both Java and Python adapters in Release 1; pilot/gate references removed; SQL lineage moved to Release 4 requirement
1.3	2026-10-29	SaaS multi-tenancy: §9.1 shared schema + forced row-level security, tenant-bound sessions and jobs, per-tenant evidence prefixes and keys; release mapping updated

End of Technical Framework Document